UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Programación 2

Manual Técnico

Fase 3 — Cliente Android

Arquitectura MVC, interfaz móvil e integración con la API REST

Ashlee Stefani Ordóñez Pérez

Carné 3190-25-4272

Tutorial público en GitHub
https://github.com/ashlee527/proyectoprogra/blob/main/fase-3/tutorial.md

- Arquitectura MVC del cliente Android
- Interfaz: menú, lección activa, editor y consola
- Integración con la API REST de la Fase 2
- Compilación y ejecución con guali-dev.jar y JDK 25
- Instalación, pruebas y publicación en GitHub

Guatemala, noviembre de 2026

Contenido

- 1. Introducción
- 2. Objetivos
- 3. Tecnologías utilizadas
- 4. Arquitectura de la solución
- 5. Estructura del proyecto
- 6. Interfaz de la aplicación
- 7. Integración con la API REST
- 8. Modelo de datos del cliente
- 9. Instalación y configuración
- 10. Compilación y ejecución
- 11. Tutorial público en GitHub
- 12. Pruebas y verificaciones
- 13. Conclusiones y recomendaciones
- Anexo A. Endpoints utilizados
- Anexo B. Solución de problemas

1. Introducción

La Fase 2 del proyecto de Programación 2 entregó una API REST construida con Spring Boot que centraliza la persistencia en MySQL y los servicios de compilación y ejecución de código Java con guali-dev.jar. Sobre esa base, la Fase 3 incorpora un cliente Android que permite al estudiante abrir lecciones, leer su contenido, escribir código, compilarlo y ejecutarlo desde un teléfono o emulador.

El cliente no se conecta a MySQL ni compila código localmente. Toda operación de datos y toda ejecución se realizan en el servidor de la Fase 2 a través de HTTP y JSON. Esta separación mantiene una única fuente de verdad para las lecciones, el historial y la configuración del JDK, y permite que el teléfono funcione incluso sin un entorno de desarrollo Java instalado.

Este documento describe la arquitectura, la interfaz, la integración con la API, la instalación, la compilación y las pruebas del cliente Android. Incluye además la URL pública del tutorial en Markdown solicitada por el enunciado.

1.1 Propósito

Documentar de forma completa y reproducible la solución de la Fase 3, de manera que el catedrático y cualquier desarrollador puedan comprender la arquitectura, instalar el proyecto, ejecutarlo y verificar su funcionamiento sin asistencia del autor.

1.2 Alcance

- Cliente Android en Java con Android Studio, Gradle y Material 3.
- Barra de menú, título de la lección activa, panel de la lección, editor de código, consola y botones Compilar y Ejecutar.
- Consumo de la API REST de la Fase 2 para lecciones, historial, configuración del JDK y ejecución de código.
- Diseño adaptable a orientación vertical y horizontal.
- Modo de demostración sin conexión con un catálogo local de cuatro lecciones.
- Pruebas unitarias de la lógica de objetivos.
- Tutorial paso a paso publicado en GitHub con capturas de pantalla.

> La Fase 3 no incluye autenticación de usuarios ni publicación en Google Play. La aplicación está pensada para ejecutarse en la red local del estudiante o del laboratorio.

2. Objetivos

2.1 Objetivo general

Implementar el front-end Android del proyecto de Programación 2 aplicando el patrón Modelo-Vista-Controlador, consumiendo la API REST de la Fase 2 y ofreciendo un entorno móvil para abrir lecciones, compilar y ejecutar código Java en el servidor.

2.2 Objetivos específicos

- Construir una interfaz que incluya barra de menú, título de la lección activa, panel de la lección, editor de código, consola y los botones Compilar y Ejecutar.
- Consumir los endpoints de lecciones, historial, configuración y ejecución de la API de la Fase 2.
- Ejecutar las peticiones HTTP fuera del hilo principal y actualizar la interfaz en el hilo de UI.
- Evaluar el objetivo de cada lección y registrar su finalización en el historial.
- Ofrecer un modo de demostración cuando la API no esté disponible.
- Documentar la instalación, el uso y las pruebas en un tutorial público de GitHub y en este manual técnico.

3. Tecnologías utilizadas

Tecnología	Versión	Uso en el proyecto
Java	25	JDK del servidor y lenguaje del proyecto
Android Studio	Versión estable reciente	Entorno de desarrollo Android
Gradle	8.9	Sistema de construcción del proyecto
Android Gradle Plugin	8.7.3	Integración de Android con Gradle
Android SDK	API 35 (Android 15)	Compilación y emulación
SDK mínimo	API 24 (Android 7.0)	Dispositivos compatibles
Java (código Android)	17	sourceCompatibility y targetCompatibility
Material 3	1.12.0	Componentes visuales
AppCompat	1.7.0	Compatibilidad de actividades
RecyclerView	1.3.2	Listado del historial
Gson	2.11.0	Serialización y deserialización JSON
HttpURLConnection	Incluido en Android	Cliente HTTP sin dependencias externas
JUnit	4.13.2	Pruebas unitarias
guali-dev.jar	1.0.0	Compilación y ejecución en el servidor
API REST Fase 2	Spring Boot 4.1.1	Back end de datos y ejecución
MySQL	8.0	Persistencia del back end
Git y GitHub	2.55.0	Control de versiones y publicación
IntelliJ IDEA	2026.2.2	Desarrollo del back end

El código Android se compila con Java 17 porque es la versión estable que soporta el Android Gradle Plugin 8.7.3. El JDK 25 se utiliza en el servidor de la Fase 2, que es quien ejecuta javac y java a través de guali-dev.jar. Esta diferencia no afecta al estudiante: el código que escribe se compila en el servidor con Java 25.

3.1 Justificación de Java y Android nativo

Se eligió Java porque es el lenguaje del curso y de las fases anteriores. Android nativo con Views permite construir la interfaz solicitada sin introducir Kotlin ni Jetpack Compose, mantiene continuidad con el cliente Swing de la Fase 2 y facilita la reutilización de los modelos y de la lógica de evaluación de objetivos.

3.2 Justificación de HttpURLConnection y Gson

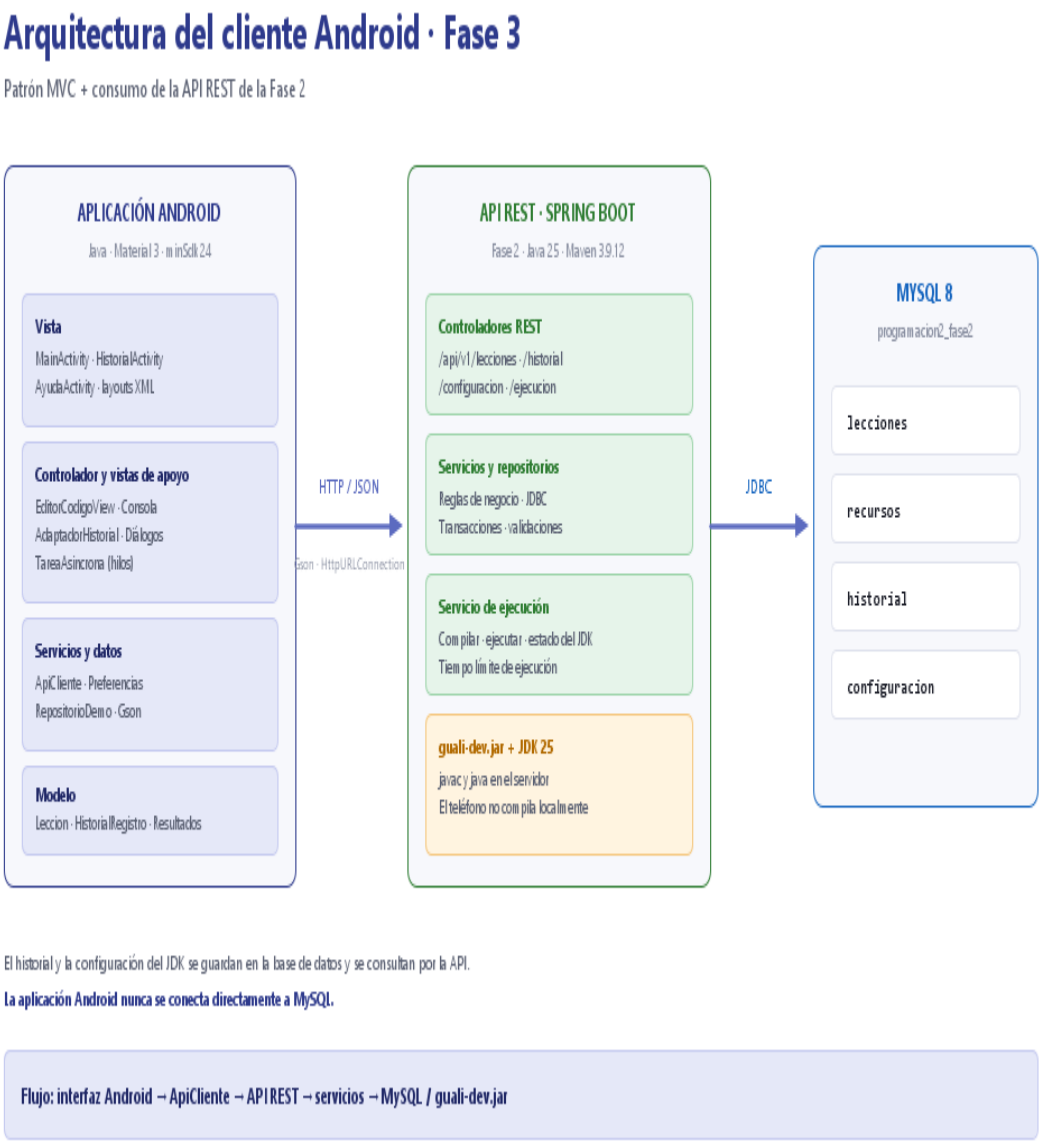
HttpURLConnection viene incluido en Android y evita agregar una biblioteca HTTP adicional. Gson convierte los objetos JSON de la API en clases Java con una configuración mínima. Ambos mantienen el APK liviano y son suficientes para el alcance del proyecto.

3.3 Justificación del modo de demostración

La compilación y la ejecución requieren el servidor, pero la interfaz debe poder revisarse aunque la API no esté iniciada. Por eso se incluye un catálogo local en assets/demo/lecciones.json que permite recorrer la pantalla, abrir lecciones y ver el editor. En ese modo los botones de compilación y ejecución permanecen deshabilitados.

4. Arquitectura de la solución

El cliente aplica el patrón Modelo-Vista-Controlador y se comunica con la API REST de la Fase 2 mediante HTTP y JSON. La API es el único componente que accede a MySQL y el único que ejecuta guali-dev.jar con el JDK 25.



Arquitectura del cliente Android

4.1 Capa de vista

Está formada por MainActivity, HistorialActivity y AyudaActivity, junto con los layouts XML, EditorCodigoView, Consola, AdaptadorHistorial y DialogoConfiguracion. La vista construye los componentes, muestra los datos y notifica al controlador las acciones del usuario. No contiene llamadas HTTP ni SQL.

4.2 Capa de control

MainActivity coordina el flujo principal: carga las lecciones al iniciar, responde a los menús y botones, invoca ApiCliente a través de TareaAsincrona y actualiza el estado de la interfaz. HistorialActivity y AyudaActivity controlan sus propias pantallas. DialogoConfiguracion coordina la configuración de la API y del JDK.

4.3 Capa de modelo

Los paquetes modelo y util contienen las clases que representan el dominio: Leccion, LeccionResumen, HistorialRegistro, ResultadoCompilacion, ResultadoEjecucion, EstadoJdk, ObjetivoLeccion y EstadoLeccion. También incluyen EvaluadorObjetivos, que decide cuándo se cumple el objetivo de una lección, y Formato, que presenta fechas y horas.

4.4 Capa de servicios y datos

ApiCliente encapsula las peticiones HTTP, Preferencias guarda la dirección de la API en SharedPreferences, RepositorioDemo carga el catálogo local y TareaAsincrona ejecuta las operaciones en un hilo secundario. MarcadorMarkdown convierte el contenido Markdown de las lecciones en texto con formato para mostrarlo en la interfaz.

4.5 Flujo de una compilación

1. El usuario abre una lección y presiona Compilar.
2. MainActivity valida que haya lección, que no esté en modo demostración, que el JDK esté configurado y que el editor tenga código.
3. TareaAsincrona ejecuta ApiCliente.compilar en un hilo secundario.
4. ApiCliente envía POST /api/v1/ejecucion/compilar con el código en JSON.
5. La API ejecuta guali-dev.jar con el JDK 25 del servidor y devuelve el resultado.
6. MainActivity muestra el resultado en la consola, habilita Ejecutar si fue correcto y evalúa el objetivo de la lección.
7. Si el objetivo se cumple, se actualiza el historial a COMPLETADA mediante PATCH /api/v1/historial/{id}.

4.6 Decisiones de diseño

- No compilar en el teléfono: Android no incluye un JDK y la API ya ofrece el servicio de ejecución.
- Separar ApiCliente de las actividades para facilitar pruebas y cambios de transporte.
- Usar TareaAsincrona con ExecutorService y Handler para no bloquear el hilo principal.
- Guardar la dirección de la API en SharedPreferences para cambiarla sin recompilar.
- Usar 10.0.2.2 como dirección por omisión en el emulador.
- Declarar usesCleartextTraffic y network_security_config para permitir HTTP local durante el desarrollo.
- Ofrecer layout vertical y horizontal con los mismos identificadores.
- Incluir un catálogo de demostración para revisar la interfaz sin servidor.
- Invalidar la compilación cuando el código cambia después de compilar.

5. Estructura del proyecto

```
fase 3/
  android/                          Proyecto Android Studio
    build.gradle                    Configuración raíz y versión de AGP
    settings.gradle                 Repositorios y módulo app
    gradle.properties               Propiedades de Gradle
    gradlew / gradlew.bat           Wrapper de Gradle 8.9
    gradle/wrapper/                 Distribución del wrapper
    local.properties.example        Ejemplo de ruta del SDK
    app/
      build.gradle                  Configuración del módulo
      proguard-rules.pro
      src/main/
        AndroidManifest.xml
        assets/demo/lecciones.json Catálogo sin conexión
        java/gt/edu/umg/practicacodigo/
          MainActivity.java
          HistorialActivity.java
          AyudaActivity.java
          api/                      ApiCliente y ApiExcepcion
          datos/                    RepositorioDemo
          modelo/                   POJO y enums
          util/                     Preferencias, Formato, Markdown, tareas
          vista/                    Editor, consola, adaptador y diálogo
        res/
          layout/                   activity_main, historial, ayuda, diálogo
          layout-land/              activity_main horizontal
          drawable/                 Íconos y fondos de chips
          mipmap-*/                 Ícono de lanzador
          raw/ayuda.md              Texto de la pantalla de ayuda
          values/                   Colores, cadenas, temas y dimensiones
          xml/                      Configuración de red
      src/test/                     EvaluadorObjetivosTest
  docs/
    img/                            Capturas y diagrama
```

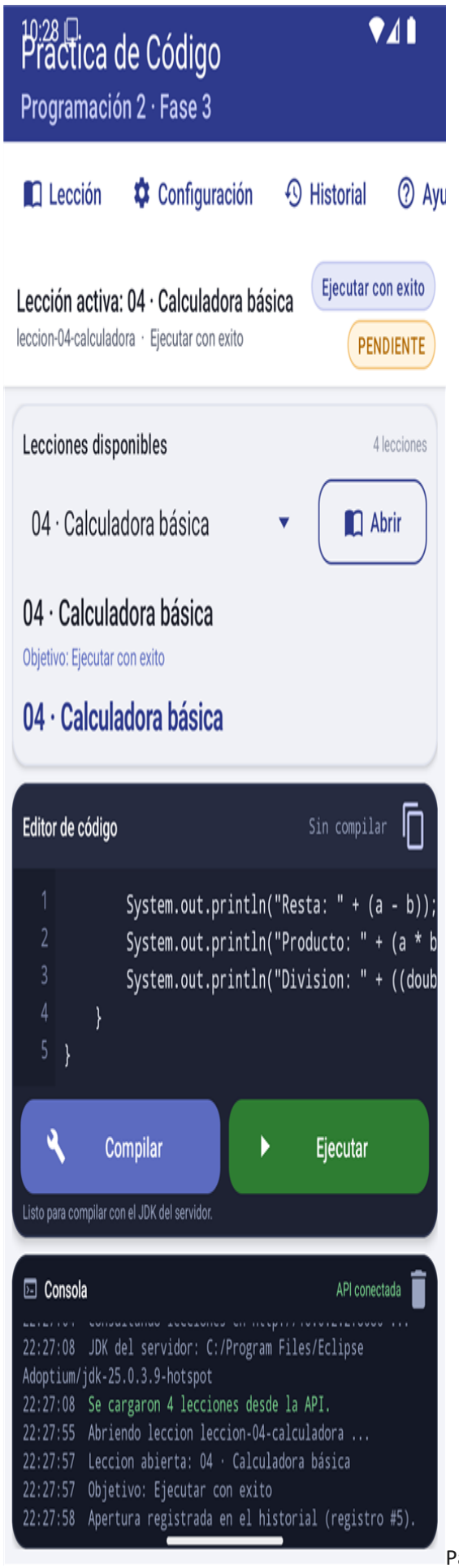
```
  Manual-Tecnico-Fase-3.pdf
tutorial.md                     Tutorial paso a paso
publicar-github.ps1             Script de publicación
README.md
```

5.1 Responsabilidad de los paquetes

Paquete	Responsabilidad
gt.edu.umg.practicacodigo	Actividades principales y coordinación
.api	Cliente HTTP y excepción de la API
.datos	Catálogo local de demostración
.modelo	Objetos JSON, enums y cuerpos de petición
.util	Preferencias, formato, Markdown, tareas y evaluación
.vista	Editor, consola, adaptador y diálogo de configuración

6. Interfaz de la aplicación

La pantalla principal se organiza en cinco franjas: barra de herramientas, barra de menú, título de la lección activa, panel de la lección con editor y consola. Los botones Compilar y Ejecutar están dentro de la tarjeta del editor para mantener el flujo de trabajo.



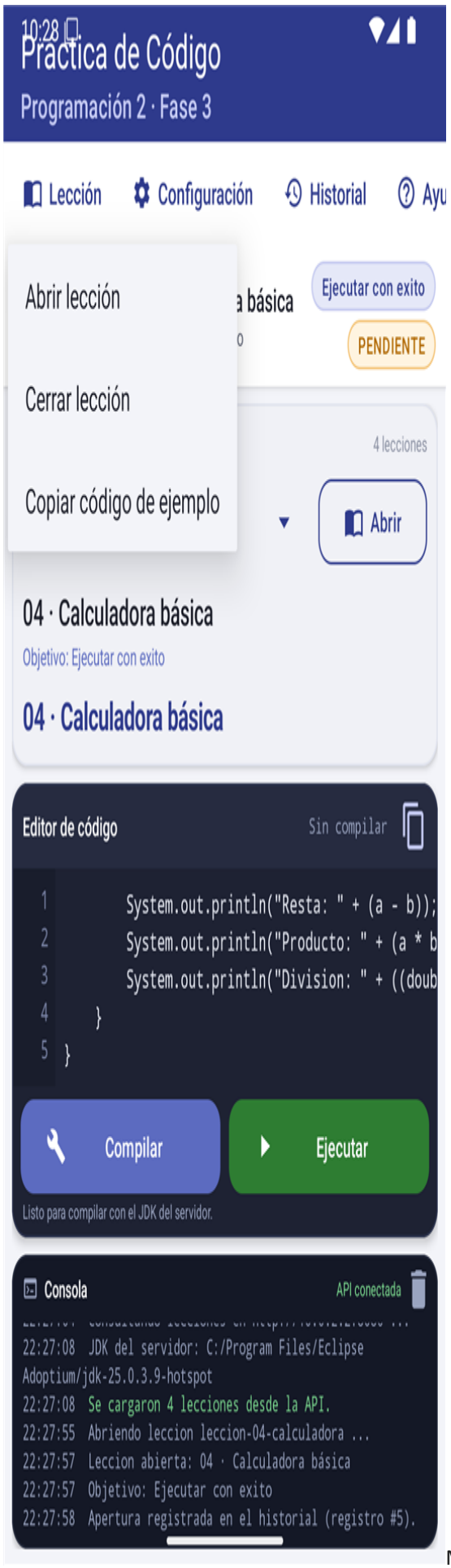
Pantalla principal de Práctica de Código

6.1 Barra de menú

La barra de menú es horizontal y contiene cuatro opciones: Lección, Configuración, Historial y Ayuda. Cada opción abre un PopupMenu con las acciones correspondientes.

Menú	Acciones
Lección	Abrir lección, cerrar lección, copiar código de ejemplo
Configuración	Configurar JDK, dirección de la API, recargar lecciones
Historial	Ver historial de sesión
Ayuda	Ayuda de la aplicación, acerca de

La barra se implementa con TextViews dentro de un HorizontalScrollView, de modo que se adapta a pantallas angostas. Los ítems tienen ícono, texto y un fondo resaltado cuando el menú está activo.



Menú Lección desplegado

6.2 Título de la lección activa

Debajo de la barra de menú se muestra el título de la lección abierta con el prefijo "Lección activa:". A la derecha se presentan dos chips: el objetivo de la lección y el estado PENDIENTE o COMPLETADA. Cuando no hay lección, el título indica "Ninguna lección abierta" y el subtítulo invita a seleccionar una.

6.3 Panel de la lección activa

El panel contiene el listado de lecciones en un Spinner, el botón Abrir, el título, el objetivo y el contenido Markdown. El contenido se renderiza con MarcadorMarkdown, que reconoce títulos, listas, negritas, código en línea y bloques de código.

El código de ejemplo se extrae del primer bloque ```java del Markdown y se copia al editor al abrir la lección. El botón de copiar permite restaurarlo si el estudiante lo modificó.

6.4 Editor de código

EditorCodigoView combina un TextView con los números de línea, un HorizontalScrollView y un EditText monoespaciado. El editor permite desplazamiento horizontal y vertical, mantiene la numeración sincronizada y notifica los cambios para invalidar la compilación anterior.

El encabezado del editor muestra el nombre de la clase compilada o el texto "Sin compilar". El botón de copiar código restaura el ejemplo de la lección.

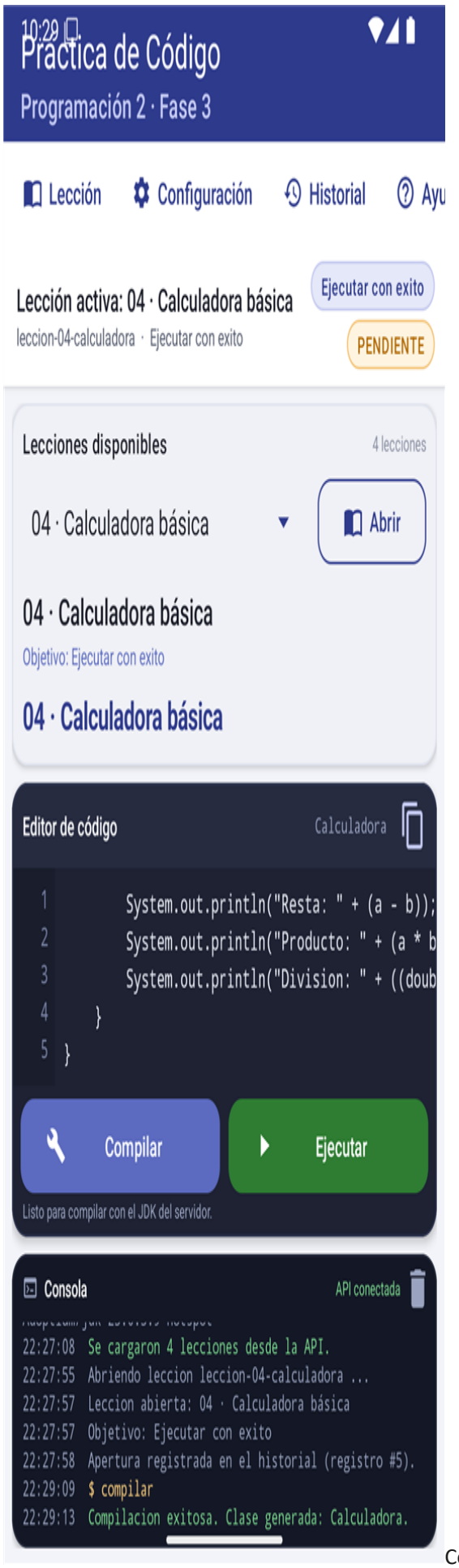
6.5 Consola

La consola es un TextView monoespaciado dentro de un ScrollView. Consola agrega cada mensaje con la hora y un color según el tipo: información, éxito, error, comando y salida del programa. La consola se desplaza automáticamente al final y puede limpiarse con el botón correspondiente.

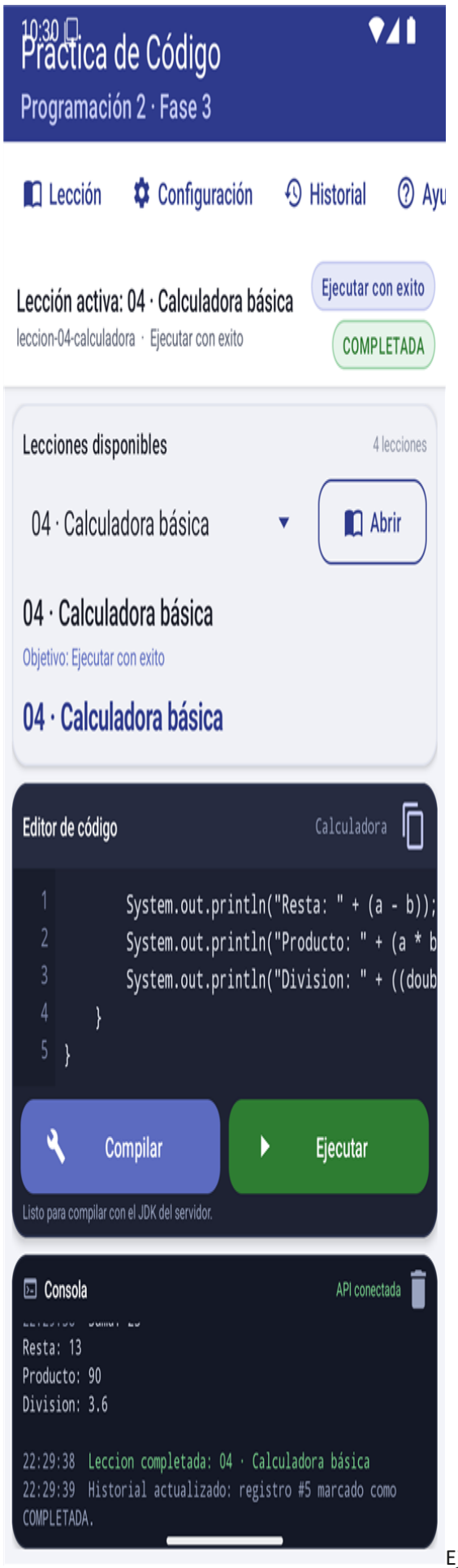
6.6 Botones Compilar y Ejecutar

Compilar envía el código a la API y muestra el resultado. Si la compilación es correcta, guarda el nombre de la clase y habilita Ejecutar. Si el código cambia, la compilación se invalida y Ejecutar se deshabilita hasta compilar de nuevo.

Ejecutar llama a la API con el nombre de la clase compilada y un tiempo límite de 10 segundos. La salida y los errores se muestran en la consola. Un tiempo de espera agotado se informa como error de ejecución.



Compilación exitosa en la consola



Ejecución y salida del programa

6.7 Diseño horizontal

En orientación apaisada se usa res/layout-land/activity_main.xml. Los identificadores son los mismos, por lo que MainActivity no cambia. El panel de la lección ocupa la columna izquierda y el editor con la consola la columna derecha.



Diseño en orientación horizontal

6.8 Historial de sesión

HistorialActivity consulta GET /api/v1/historial, ordena los registros por fecha descendente y muestra el resumen "N registros · M completadas". Cada fila incluye la lección, la fecha de apertura, el objetivo, el detalle y un chip de estado. El botón Actualizar recarga la lista.



Historial de sesión

6.9 Configuración del entorno

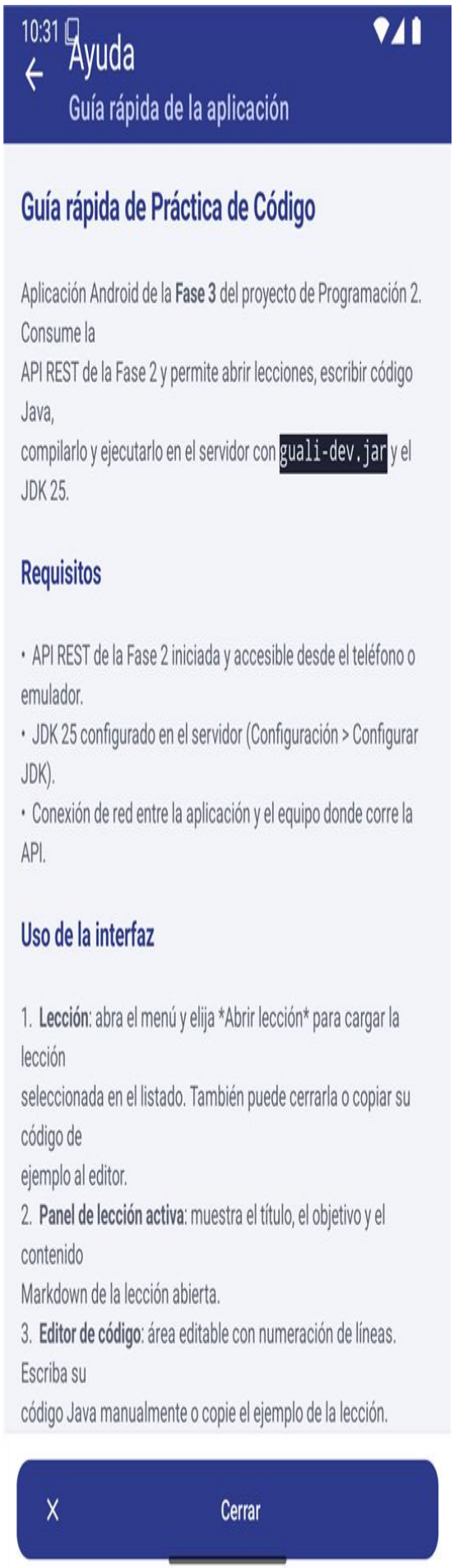
El diálogo de configuración permite consultar el estado del JDK del servidor, guardar la ruta del JDK 25 y cambiar la dirección de la API. La ruta se guarda en la API con la clave jdk.ruta y la usa guali-dev.jar. El teléfono no puede explorar el sistema de archivos del servidor, por eso la ruta se escribe manualmente.



Configuración de la API y del JDK

6.10 Pantalla de ayuda

AyudaActivity lee res/raw/ayuda.md y lo muestra con el mismo renderizador Markdown de las lecciones. El texto incluye requisitos, uso de la interfaz, objetivos, configuración, modo de demostración y solución de problemas.



Pantalla de ayuda

7. Integración con la API REST

ApiCliente encapsula las peticiones con HttpURLConnection y Gson. Todos los métodos son síncronos y se ejecutan desde TareaAsincrona en un hilo secundario. La dirección base se normaliza para eliminar la barra final y se obtiene de Preferencias.

7.1 Endpoints utilizados

Método	Ruta	Uso en el cliente
GET	/api/v1/lecciones	Cargar el listado de lecciones
GET	/api/v1/lecciones/{id}	Abrir una lección completa
GET	/api/v1/historial	Mostrar el historial
POST	/api/v1/historial	Registrar la apertura de una lección
PATCH	/api/v1/historial/{id}	Marcar la lección como COMPLETADA
GET	/api/v1/ejecucion/jdk	Consultar el estado del JDK
PUT	/api/v1/configuracion/jdk.ruta	Guardar la ruta del JDK
POST	/api/v1/ejecucion/compilar	Compilar el código del editor
POST	/api/v1/ejecucion/ejecutar	Ejecutar la clase compilada

7.2 Tiempos de espera

Operación	Tiempo de conexión	Tiempo de lectura
Conexión general	10 segundos	20 segundos
Compilación	10 segundos	150 segundos
Ejecución	10 segundos	90 segundos

La compilación tiene un tiempo de lectura mayor porque el servidor debe iniciar javac y resolver las dependencias de guali-dev.jar. La ejecución tiene un tiempo límite adicional de 10 segundos configurado en la solicitud.

7.3 Formato de las solicitudes

Registrar una apertura:

```
{
```

```
  "leccionId": 1,
  "leccionNombre": "leccion-01-hola-mundo",
  "objetivo": "COMPILAR_EXITOSO",
  "detalle": null
}
```

Compilar:

```
{
  "codigo": "public class HolaMundo { public static void main(String[] args) {
System.out.println(\"Hola\"); } }"
}
```

Ejecutar:

```
{
  "nombreClase": "HolaMundo",
  "timeoutSegundos": 10
}
```

Marcar como completada:

```
{
  "estado": "COMPLETADA",
  "detalle": "Se cumplio el objetivo COMPILAR_EXITOSO al compilar."
}
```

7.4 Manejo de errores

ApiCliente lee el cuerpo de error de la API y construye una ApiExcepcion con el código HTTP y el mensaje. Si no hay conexión, informa "No se pudo conectar con <dirección>". MainActivity muestra el mensaje en la consola y, cuando corresponde, activa el modo de demostración.

7.5 Configuración del JDK

La ruta del JDK se guarda en la tabla configuracion del servidor. Al abrir el diálogo, el cliente consulta GET /api/v1/ejecucion/jdk. Si el JDK está configurado, muestra la ruta; si no, solicita escribirla. El botón Verificar consulta el estado y el botón Guardar envía la ruta con PUT /api/v1/configuracion/jdk.ruta.

7.6 Historial y objetivos

Al abrir una lección se registra una apertura en estado PENDIENTE. Cuando la compilación o la ejecución cumplen el objetivo configurado, el cliente envía PATCH con estado COMPLETADA y un detalle descriptivo. Si la lección ya estaba completada, no se vuelve a actualizar.

Objetivo	Se cumple cuando
COMPILAR_EXITOSO	La compilación termina sin errores

COMPILAR_CON_ERROR	La compilación reporta errores
EJECUTAR_EXITOSO	La ejecución termina sin errores ni tiempo agotado
EJECUTAR_CON_ERROR	La ejecución falla o excede el tiempo límite

8. Modelo de datos del cliente

Los modelos son clases Java simples que Gson convierte desde el JSON de la API. Los nombres de los campos coinciden con el contrato OpenAPI de la Fase 2.

Clase	Uso
LeccionResumen	Datos del listado: id, nombre, título y objetivo
Leccion	Lección completa: contenido Markdown y recursos
RecursoLeccion	Metadatos de una imagen asociada
HistorialRegistro	Registro del historial y su estado
EstadoJdk	Ruta y estado de configuración del JDK
ResultadoCompilacion	Éxito, salida y nombre de la clase
ResultadoEjecucion	Éxito, salida, errores y tiempo agotado
SolicitudCompilacion	Cuerpo con el código a compilar
SolicitudEjecucion	Cuerpo con la clase y el tiempo límite
SolicitudHistorial	Cuerpo para registrar una apertura
ActualizacionHistorial	Cuerpo para cambiar estado y detalle
ValorConfiguracion	Cuerpo con el valor de una clave
ErrorRespuesta	Cuerpo uniforme de error de la API
ObjetivoLeccion	Enum de objetivos con etiquetas legibles
EstadoLeccion	Enum COMPLETADA y PENDIENTE

LeccionResumen no incluye el campo recursos. El listado de la API devuelve un entero con el número de recursos, mientras que el detalle devuelve un arreglo. Gson ignora el entero del resumen y mapea el arreglo al campo recursos de Leccion, lo que evita conflictos de herencia.

9. Instalación y configuración

9.1 Requisitos previos

- JDK 25 instalado y JAVA_HOME configurado en el servidor.
- Android Studio con Android SDK 35 y Build-Tools 35.
- Emulador con imagen API 35 o dispositivo físico con depuración USB.
- API REST de la Fase 2 en ejecución.
- MySQL 8 con la base de datos de la Fase 2.
- Git 2.55 o superior para publicar el tutorial.

9.2 Verificación del entorno

Herramienta	Comando	Resultado esperado
JDK	java -version	Versión 25.x
Compilador	javac -version	Versión 25.x
Gradle wrapper	gradlew --version	Gradle 8.9
Android SDK	SDK Manager	Platform 35 instalada
Maven	mvn -version	3.9.12 o superior
MySQL	mysql --version	8.0.x
Git	git --version	2.55.x

9.3 Abrir el proyecto

1. Inicie Android Studio.
2. Elija Open y seleccione la carpeta android del proyecto.
3. Espere la sincronización de Gradle.
4. Si aparece el aviso de SDK, copie local.properties.example a local.properties y ajuste sdk.dir con la ruta del SDK.
5. Verifique en SDK Manager que esté instalada la plataforma 35.

9.4 Iniciar la API de la Fase 2

Desde la carpeta backend de la Fase 2:

```
java -jar target\proyecto-fase-2-backend.jar
```

Verifique la API:

```
curl.exe http://localhost:8080/api/v1/salud
```

La API debe devolver estado OK y la base de datos disponible. Si la API no está iniciada, la aplicación funciona en modo de demostración, pero no puede compilar ni ejecutar.

9.5 Configurar la dirección de la API

En el emulador, la dirección por omisión es http://10.0.2.2:8080. En un dispositivo físico debe usarse la IP del equipo en la red local, por ejemplo http://192.168.1.20:8080. La dirección se cambia desde Configuración > Dirección de la API y se guarda en SharedPreferences.

9.6 Configurar el JDK del servidor

1. Abra Configuración > Configurar JDK.
2. Escriba la ruta absoluta del JDK 25 en el servidor. Ejemplo: C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot.
3. Presione Verificar para consultar el estado.
4. Presione Guardar. La ruta se envía a la API y queda disponible para guali-dev.jar.

> La ruta debe existir en el equipo donde corre la API, no en el teléfono. El teléfono no puede explorar el sistema de archivos del servidor.

10. Compilación y ejecución

10.1 Compilar el APK

Desde la carpeta android:

```
.\gradlew.bat assembleDebug
```

El APK se genera en app/build/outputs/apk/debug/app-debug.apk. Para una versión de entrega:

```
.\gradlew.bat assembleRelease
```

10.2 Ejecutar en el emulador

1. Abra Device Manager y cree un dispositivo virtual con API 35.
2. Inicie el emulador.
3. Presione Run o use Shift + F10.
4. Verifique que la consola muestre "API conectada" y las lecciones cargadas.
5. Abra una lección, presione Compilar y luego Ejecutar.

10.3 Ejecutar en un dispositivo físico

1. Habilite las opciones de desarrollador y la depuración USB.
2. Conecte el teléfono por USB y autorice el equipo.
3. Averigüe la IP del equipo con ipconfig.
4. Cambie la dirección de la API a http://IP_DEL_EQUIPO:8080.
5. Permita el puerto 8080 en el firewall del equipo.
6. Ejecute la aplicación desde Android Studio.

10.4 Flujo de uso

1. Abra la aplicación y espere la carga de lecciones.
2. Seleccione una lección en el listado y presione Abrir.
3. Revise el título, el objetivo y el contenido en el panel.
4. Modifique el código en el editor si es necesario.
5. Presione Compilar y revise la consola.
6. Si la compilación fue correcta, presione Ejecutar.
7. Al cumplir el objetivo, la lección se marca COMPLETADA.
8. Consulte el resultado en Historial.

10.5 Pruebas unitarias

Desde la carpeta android:

```
.\gradlew.bat test
```

El reporte queda en app/build/reports/tests/test/index.html. Las pruebas de EvaluadorObjetivos no requieren emulador porque no dependen del SDK de Android.

11. Tutorial público en GitHub

El enunciado solicita una URL pública y funcional de un tutorial en Markdown con capturas de pantalla. El tutorial de esta entrega está publicado en:

```
https://github.com/ashlee527/proyectoprogra/blob/main/fase-3/tutorial.md
```

11.1 Contenido del tutorial

El tutorial incluye, en orden:

- Presentación del proyecto y de la arquitectura.
- Requisitos previos e instalación de JDK 25, Android Studio, SDK y Gradle.
- Creación del proyecto en Android Studio.
- Configuración de Gradle, manifiesto, permisos y red local.
- Recursos visuales, layouts vertical y horizontal.
- Modelo de datos, ApiCliente, preferencias y tareas asíncronas.
- Editor con numeración, consola y lógica de MainActivity.
- Historial, ayuda, configuración del JDK y modo de demostración.
- Pruebas unitarias, compilación, ejecución y publicación en GitHub.
- Solución de problemas y resumen de la interfaz.

11.2 Capturas incluidas

El tutorial referencia las nueve imágenes de docs/img: pantalla principal, menú Lección, compilación exitosa, ejecución, historial, configuración del JDK, ayuda, orientación horizontal y arquitectura. GitHub las muestra directamente en el navegador porque están en el mismo repositorio.

> Las imágenes son capturas reales de la aplicación ejecutándose en un emulador Android 15 (API 35, 1080 × 1920), conectado a la API de la Fase 2 en http://10.0.2.2:8080. Muestran las cuatro lecciones importadas, la compilación con guali-dev.jar y la ejecución de la clase Calculadora en el servidor.

11.3 Publicación del repositorio

El repositorio debe ser público y contener la carpeta fase-3 en la rama main. El proyecto incluye el script publicar-github.ps1, que:

1. Verifica que Git esté instalado.
2. Copia el proyecto a una carpeta llamada fase-3.
3. Excluye build, .gradle, .idea, local.properties y archivos zip.
4. Inicializa el repositorio si es necesario.
5. Crea el commit y la rama main.
6. Configura el remoto https://github.com/ashlee527/proyectoprogra.git.
7. Muestra el comando de publicación o ejecuta el push con la opción -Push.

Ejecución:

```
cd "C:\Users\asop2\Downloads\fase 3"

.\publicar-github.ps1
```

Para publicar directamente:

```
.\publicar-github.ps1 -Push
```

11.4 Verificación de la URL

Antes de entregar, compruebe que:

- El repositorio sea público.
- La rama se llame main.
- El archivo se llame exactamente tutorial.md.
- La ruta sea fase-3/tutorial.md.
- Las imágenes de docs/img se muestren en el navegador.
- La URL abra el tutorial sin iniciar sesión.

Si el repositorio aún no existe, créelo en GitHub con el nombre proyectoprogra, visibilidad Public, con un README inicial, y luego ejecute el script.

12. Pruebas y verificaciones

12.1 Pruebas unitarias

Se incluye la clase EvaluadorObjetivosTest con seis pruebas que verifican las combinaciones de objetivo y resultado:

Prueba	Verifica
compilarExitosoCumpleSoloConCompilacionCorrecta	COMPILAR_EXITOSO solo con éxito
compilarConErrorCumpleSoloConCompilacionFallida	COMPILAR_CON_ERROR solo con fallo
objetivosDeEjecucionNoSeCumplenAlCompilar	Los objetivos de ejecución no se cumplen al compilar
ejecutarExitosoCumpleSoloSinErroresNiTimeout	EJECUTAR_EXITOSO sin errores ni timeout
ejecutarConErrorCumpleConFalloOTimeout	EJECUTAR_CON_ERROR con fallo o timeout
objetivoNuloNuncaCumple	Un objetivo nulo nunca se cumple

12.2 Verificaciones realizadas

La entrega se compiló y ejecutó en un equipo con Android Studio 2026.2.2.8, Android SDK 35, Build-Tools 35.0.0 y Gradle 8.9. Resultados obtenidos:

Verificación	Comando o entorno	Resultado
Compilación del APK	.\gradlew.bat assembleDebug	BUILD SUCCESSFUL; app-debug.apk de 5.75 MB
Pruebas unitarias	.\gradlew.bat testDebugUnitTest	6 pruebas, 0 fallos, 0 errores
Instalación	adb install -r app-debug.apk	Success
Arranque	Emulador Pixel_2_API_35, Android 15 (API 35)	Aplicación abierta sin errores
Conexión con la API	http://10.0.2.2:8080	4 lecciones cargadas desde MySQL
Apertura de lección	Lección 04 · Calculadora básica	Título, objetivo, contenido y editor visibles
Compilación remota	POST /api/v1/ejecucion/compilar	Compilacion exitosa. Clase generada: Calculadora.

Ejecución remota	POST /api/v1/ejecucion/ejecutar	Suma: 23; Resta: 13; Producto: 90; Division: 3.6
Objetivo e historial	Lección 04	Estado COMPLETADA y registro actualizado en el historial
Capturas	docs/img	8 capturas reales del emulador y 1 diagrama de arquitectura

12.3 Alcance de la verificación

Las pruebas se realizaron en el emulador oficial Pixel_2_API_35 con Android 15 (API 35), resolución 1080 x 1920 y 1536 MB de RAM. La API de la Fase 2 se ejecutó en el mismo equipo con MySQL en el puerto 3307 y el JDK 25 configurado para guali-dev.jar. No se probó en un dispositivo físico ni con una API desplegada en Internet; para ese caso deben ajustarse la dirección de la API y la configuración de red segura.

El flujo completo se verificó con datos reales: abrir lección, compilar, ejecutar, cumplir el objetivo y consultar el historial. Las capturas del manual y del tutorial corresponden a esa ejecución, no a maquetas.

12.4 Pruebas manuales recomendadas

Caso	Resultado esperado
API disponible	Se cargan las lecciones y la consola indica API conectada
API no disponible	Se activa el modo de demostración y se deshabilitan Compilar y Ejecutar
Abrir lección	Se muestran título, objetivo, contenido y código de ejemplo
Compilar correcto	La consola muestra éxito y se habilita Ejecutar
Compilar con error	La consola muestra el error y Ejecutar sigue deshabilitado
Ejecutar correcto	La salida del programa aparece en la consola
Ejecutar con error	La consola muestra el error o el tiempo agotado
Objetivo cumplido	La lección se marca COMPLETADA y se actualiza el historial
Historial	Se listan los registros con su estado y fecha
Configuración	Se guarda la ruta del JDK y la dirección de la API
Orientación horizontal	La interfaz muestra dos columnas sin perder funciones

13. Conclusiones y recomendaciones

13.1 Conclusiones

- La separación en tres capas permitió reutilizar la API de la Fase 2 sin duplicar la lógica de persistencia ni la ejecución de código.
- Android nativo con Java mantuvo continuidad con el curso y permitió construir la interfaz solicitada con componentes estándar.
- HttpURLConnection y Gson fueron suficientes para consumir la API y mantuvieron el APK liviano.
- TareaAsincrona evitó bloquear el hilo principal y mantuvo la interfaz fluida durante las operaciones de red.
- El modo de demostración permite revisar la interfaz aun sin servidor, sin ocultar que la compilación requiere la API.
- La evaluación de objetivos y el historial completan el ciclo de práctica propuesto por el curso.
- El tutorial público y este manual documentan la instalación, el uso y las pruebas de la entrega.

13.2 Recomendaciones

- Mantener las capturas reales del emulador actualizadas si cambia la interfaz o el catálogo de lecciones.
- Usar HTTPS y eliminar usesCleartextTraffic en un despliegue fuera de la red local.
- Agregar autenticación si la API se expone fuera del laboratorio.
- Incorporar paginación en el historial si el número de registros crece.
- Agregar pruebas instrumentadas con Espresso para los flujos de compilación y ejecución.
- Versionar el proyecto en GitHub con etiquetas por fase y mantener el tutorial actualizado.

Anexo A. Endpoints utilizados

Método	Ruta	Cuerpo	Respuesta
GET	/api/v1/salud	Ninguno	Estado de la API y la base de datos
GET	/api/v1/lecciones	Ninguno	Lista de LeccionResumen
GET	/api/v1/lecciones/{id}	Ninguno	Leccion completa
GET	/api/v1/historial	Ninguno	Lista de HistorialRegistro
POST	/api/v1/historial	SolicitudHistorial	HistorialRegistro creado
PATCH	/api/v1/historial/{id}	ActualizacionHistorial	HistorialRegistro actualizado
GET	/api/v1/ejecucion/jdk	Ninguno	EstadoJdk
PUT	/api/v1/configuracion/jdk.ruta	ValorConfiguracion	Valor guardado
POST	/api/v1/ejecucion/compilar	SolicitudCompilacion	ResultadoCompilacion
POST	/api/v1/ejecucion/ejecutar	SolicitudEjecucion	ResultadoEjecucion

Anexo B. Solución de problemas

Síntoma	Causa probable	Solución
No se pudo conectar con la API	El backend no está iniciado o la URL es incorrecta	Inicie el backend y revise Configuración > Dirección de la API
JDK no configurado	Falta la clave jdk.ruta en la API	Guarde la ruta del JDK 25 desde Configurar JDK
Compile el código actual antes de ejecutarlo	El código cambió después de compilar	Presione Compilar de nuevo
Cleartext HTTP traffic not permitted	Falta la configuración de red	Verifique usesCleartextTraffic y network_security_config.xml
El emulador no conecta con localhost	localhost apunta al emulador	Use 10.0.2.2
El dispositivo físico no conecta	IP o firewall	Use la IP de la LAN y permita el puerto 8080
Gradle no descarga dependencias	Sin Internet o proxy	Revise la conexión y el proxy de Android Studio
SDK location not found	Falta local.properties	Copie local.properties.example y ajuste sdk.dir
La aplicación se abre en modo demostración	La API no responde	Inicie la API y presione Recargar lecciones
No aparece la clase compilada	La compilación falló	Revise los errores de la consola y corrija el código

Datos de la entrega

Campo	Valor
Estudiante	Ashlee Stefani Ordóñez Pérez
Carné	3190-25-4272
Curso	Programación 2
Fase	3
Catedrático	Ing. MGSI. Cesar Hernandez
Universidad	Universidad Mariano Gálvez de Guatemala
Fecha	Guatemala, noviembre de 2026
Tutorial	https://github.com/ashlee527/proyectoprogra/blob/main/fase-3/tutorial.md